

§ Units I & II | RV University | CIE

1 Preparation

Scope: Unit I (Abstract Data Types & Memory Allocation) + Unit II (Linked Lists) **Standard:** Technically precise, exam-ready, zero fluff.

Side Remark

An array behaves like numbered pages. A linked list behaves like following references from page to page.

Traversal Hint

Array = direct indexed access. Linked list = node-by-node journey through the shadows.

§ Table of Contents

- Data Structures — Foundations

DS Formula

Data Structures + Algorithms = Program. Elegant logic still needs efficient storage.

- 1.1 Core Equation
- 1.2 Classification Taxonomy

Margin Note

Data Structures + Algorithms = Program. Good logic reads better when the underlying structure is chosen well.

- 1.3 Primitive vs Non-Primitive
- 1.4 Linear vs Non-Linear
- 1.5 Basic Operations
- 1.6 Strategies for Choosing a Data Structure
- Abstract Data Types (ADT)

- 2.1 Definition
- 2.2 The List ADT
- 2.3 List ADT Operations
- 2.4 Implementations of List ADT

◆ **Memory Reminder**

Stack is fast and scoped. Heap is flexible and manual. Bugs adore confusion between the two.

- Memory Architecture in C

Reader's Note

Stack is quick and scoped. Heap is flexible and manual. In C, memory mistakes are rarely quiet.

- 3.1 C Program Memory Layout
- 3.2 Text Segment
- 3.3 Initialized Data Segment
- 3.4 Uninitialized Data Segment (BSS)
- 3.5 Stack Memory
- 3.6 Heap Memory
- Memory Allocation in C
 - 4.1 Compile-Time (Static) Memory Allocation
 - 4.2 Run-Time (Dynamic) Memory Allocation
 - 4.3 malloc()
 - 4.4 calloc()
 - 4.5 realloc()
 - 4.6 free()
 - 4.7 malloc() vs calloc() Comparison
 - 4.8 Static vs Dynamic Allocation Comparison
- Arrays — Array-Based List Implementation
 - 5.1 Definition & Properties
 - 5.2 Array Operations — Algorithms & Code
- Linked Lists
 - 6.1 Definition & Motivation
 - 6.2 Linked List vs Array
 - 6.3 Node Representation in C
 - 6.4 Types of Linked Lists

- Singly Linked List (SLL)
 - 7.1 Structure & Pointer Model
 - 7.2 Create & Display
 - 7.3 Insert Operations
 - 7.4 Delete Operations
 - 7.5 Search Operation
 - Doubly Linked List (DLL)
 - 8.1 Structure & Pointer Model
 - 8.2 Create & Display (Forward/Backward)
 - 8.3 Insert Operations
 - 8.4 Delete Operations
 - 8.5 Search Operation
 - Circular Singly Linked List (CSLL)
 - 9.1 Structure & Invariant
 - 9.2 Create & Display
 - 9.3 Insert Operations
 - 9.4 Delete Operations
 - 9.5 Search Operation
 - Circular Doubly Linked List (CDLL)
 - 10.1 Structure & Invariant
 - 10.2 Insert Operations
 - 10.3 Delete Operations
 - 10.4 Search Operation
 - Comparison Tables — All List Types
 - Applications & Polynomial Manipulation
 - 12.1 Polynomial Representation
 - 12.2 Polynomial Addition using Linked List
 - 12.3 Polynomial Multiplication
-

§ 1. Data Structures — Foundations

● 1.1 Core Equation

Data Structures + Algorithms = Program

Term	Precise Definition
Data	A value or set of values belonging to a defined data type (e.g., int, char, float)
Structure	A scheme for organizing related pieces of information to enable efficient access and modification
Data Structure	A particular way of organizing data in a computer so that it can be used efficiently; a container that stores data in an organized form
Algorithm	Logic + Control — a finite, ordered sequence of well-defined instructions to solve a problem
Program	An executable set of instructions operating on data structures via algorithms

Note. Key Insight (Wirth's Law): The choice of data structure directly determines the efficiency of algorithms that operate on it. Poor structure choice → poor algorithm performance regardless of algorithmic optimization.

● 1.2 Classification Taxonomy

Data Structures

├ Primitive (Built-in types)

| ├ int

| ├ float

| ├ char

| └ double

└ Non-Primitive (User-defined / composite)



Visual: Data Structure Classification Diagram

- 1.3 Primitive vs Non-Primitive

Attribute	Primitive	Non-Primitive
Definition	Directly supported by the machine/language; represents a single value	Constructed from primitive types; represents a collection or relationship of values
Examples	int, float, char, double, bool	Arrays, Linked Lists, Stacks, Queues, Trees, Graphs
User-Defined?	No	Yes (can be designed by the user)
Decomposable?	No (atomic unit)	Yes (into primitive/non-primitive sub-components)
Memory	Fixed size (platform-dependent)	Variable size

- 1.4 Linear vs Non-Linear

Attribute	Linear	Non-Linear
Traversal	Sequential — elements visited in a single run	Hierarchical/networked — multiple paths possible
Memory Layout	Conceptually sequential	Non-sequential
Relationship	Each element has at most one predecessor and one successor	Each element may be attached to several others
Examples	Arrays, Linked Lists, Stacks, Queues	Trees, Graphs, Heaps
Traversal Complexity	O(n) single pass	O(n) but requires multi-path strategies (BFS, DFS)

- 1.5 Basic Operations

Operation	Definition	Example
Traversal	Process every data element exactly once	Printing all elements of an array
Insertion	Add a new data element at a specified position	insert(list, 5, position=3)
Deletion	Remove a specified data element if it exists	delete(list, key=5)
Searching	Locate a specified data element within the structure	search(list, key=5) → position 3
Sorting	Arrange data elements in a specified order (ascending/descending)	Bubble sort on array

Merging	Combine two data structures of the same type into a single new structure	Merge two sorted linked lists
----------------	--	-------------------------------

• 1.6 Strategies for Choosing a Data Structure

The choice is governed by analyzing:

- **Nature of data** — Is it homogeneous? Ordered? Keyed?
- **Operations required** — Which operations (insert/delete/search/traverse) are most frequent?
- **Frequency of operations** — Frequent insertion/deletion → Linked List; frequent random access → Array
- **Memory constraints** — Static bound known in advance → Array; dynamic/unknown bound → Linked List or dynamic structure
- **Time complexity requirements** — **O(1)** random access required → Array; **O(1)** insertion at head required → Linked List

Scenario	Recommended Structure	Reason
Fixed-size collection, frequent random access	Array	O(1) index-based access
Frequent insertion/deletion at arbitrary positions	Linked List	No shifting required; O(1) pointer update
LIFO access pattern	Stack	O(1) push/pop
FIFO access pattern	Queue	O(1) enqueue/dequeue
Hierarchical data (file systems, expression parsing)	Tree	Reflects parent-child relationships
Network/graph-based relationships	Graph	Models many-to-many connections
Key-value lookups	Hash Table	O(1) average-case access

§ 2. Abstract Data Types (ADT)

● 2.1 Definition

An **Abstract Data Type (ADT)** is:

A mathematical model of a data structure that specifies the type of data stored, the operations that can be performed on that data, and the types of parameters of the operations — **independent of any particular implementation.**

Three equivalent formulations (all are correct and may be tested):

- *"An ADT refers to a set of data values and associated operations that are specified accurately, independent of any particular implementation."*
- *"An ADT is a user-defined data type that encapsulates a range of data values and their functions."*
- *"An ADT is a mathematical model of a data structure — it describes a container which holds a finite number of objects where objects may be associated through a given binary relationship."*

Key distinction: ADT specifies **what** operations do (behavior/interface), not **how** they are implemented.

Example. ♦ Example: Stack ADT defines push, pop, peek, isEmpty — but does NOT specify whether it uses an array or linked list internally.

Common ADTs: **List, Stack, Queue, Tree, Graph, Priority Queue**

● 2.2 The List ADT

Definition. Formal Definition: A list (or sequence) is an ADT representing a countable number of ordered values where the same value may occur more than once.

Mathematical notation:

- A sequence: $A_1, A_2, A_3, \dots, A_N$
- Length of list = N
- A_1 = first element, A_N = last element, A_i = element at position i
- If $N = 0 \rightarrow$ **empty list**

- Linear order: A_i precedes A_{i+1} ; equivalently, A_i follows A_{i-1}

Properties:

- Ordered (position matters)
- Allows duplicate values (each occurrence is a distinct item)
- Countable (finite number of elements)

• 2.3 List ADT Operations

Operation	Signature	Description
get(i)	get(position) $\rightarrow$ element	Return element at position i
insert(x, i)	insert(element, position)	Insert element x at position i
remove(x)	remove(element)	Remove first occurrence of x from non-empty list
removeAt(i)	removeAt(position)	Remove element at specified position from non-empty list
replace(x, i)	replace(element, position)	Replace element at position i with x
size()	size() $\rightarrow$ int	Return current count of elements
isEmpty()	isEmpty() $\rightarrow$ bool	Return true if list contains no elements
isFull()	isFull() $\rightarrow$ bool	Return true if list has reached maximum capacity

- 2.4 Implementations of List ADT

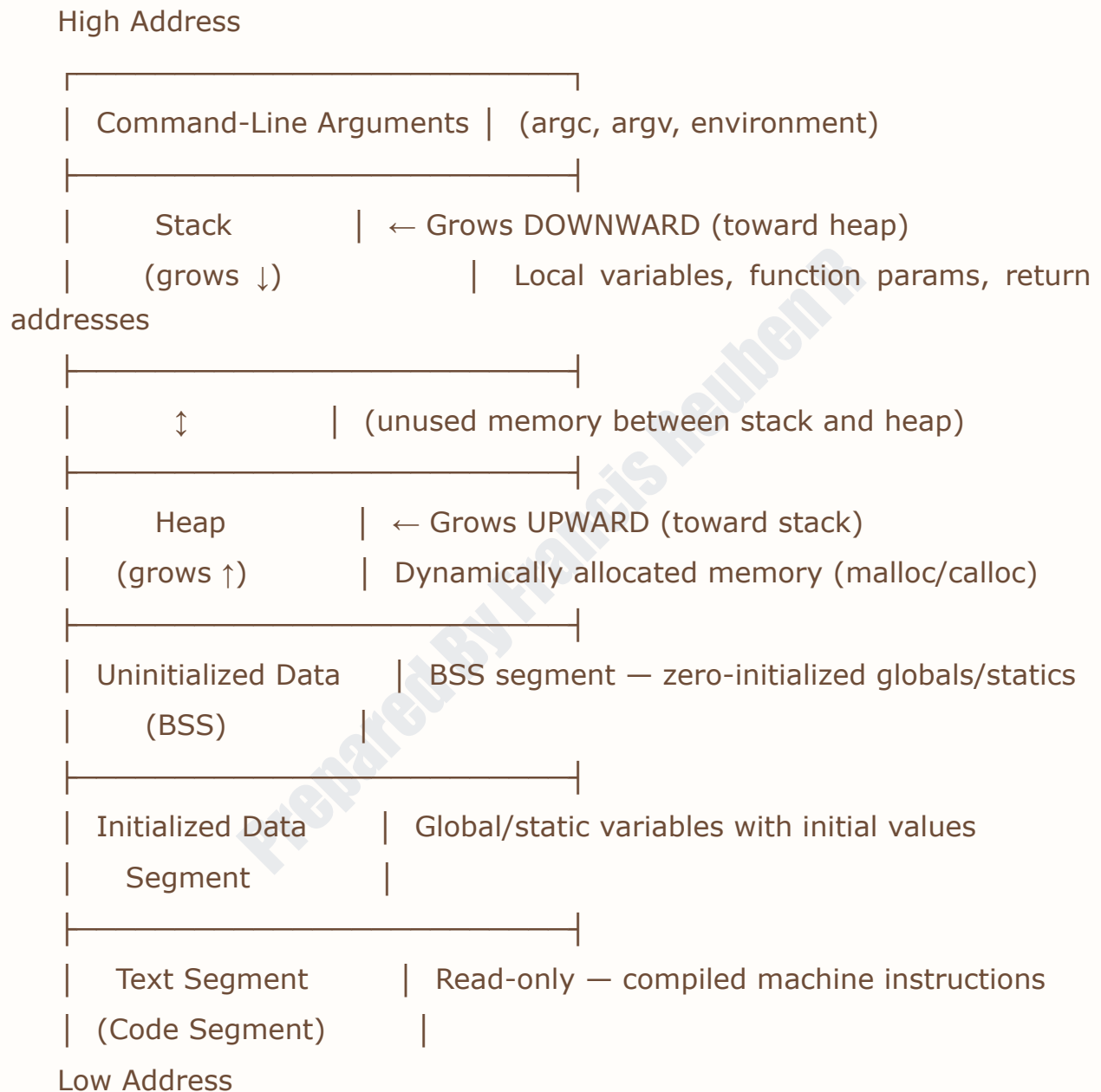
Attribute	Array-Based	Linked List-Based
Access Type	Random access — O(1) direct index access	Sequential access — must traverse from head
Memory	Contiguous; allocated at compile time (static) or malloc (dynamic)	Non-contiguous; each node allocated independently
Analogy	Book (any page openable independently)	Roll of paper/tape (must unroll to reach prior content)
Insertion/Deletion	O(n) — requires shifting elements	O(1) — pointer update only (once position found)
Memory overhead	Low (just data)	Higher (data + pointer(s) per node)

Prepared By Fico

§ 3. Memory Architecture in C

● 3.1 C Program Memory Layout

When a C program is compiled and executed, the binary .exe is loaded into RAM in an organized manner with **six distinct segments**:



Visual: C Memory Layout Diagram

Critical rule: Each segment has specific read/write permissions. Accessing a segment in a manner inconsistent with its permissions causes a **Segmentation Fault**.

- 3.2 Text Segment
 - **Contents:** Compiled binary machine instructions
 - **Permissions:** Read-only (prevents accidental modification of code)
 - **Sharing:** Shareable — a single copy in memory suffices for multiple instances of the same program (e.g., multiple terminal sessions running the same text editor)
 - **Location:** Lowest address in the memory layout
-

- 3.3 Initialized Data Segment
 - **Contents:** Global, external, static, and constant variables **whose values are explicitly initialized** at declaration
 - **Permissions:** Read-write (values can change during execution)
 - **Sub-division:**
 - **Read-write area:** Regular initialized globals and statics
 - **Read-only area:** const variables (value cannot be modified post-initialization)

Example. 💡 Example:

```
int global_var = 10;      // Initialized data segment (read-write)
const int MAX = 100;     // Initialized data segment (read-only)
static int counter = 0;   // Initialized data segment (read-write)
```

- 3.4 Uninitialized Data Segment (BSS)
 - **Full name:** Block Started by Symbol (BSS)
 - **Contents:** Global and static variables **not explicitly initialized** (or explicitly initialized to 0)
 - **Permissions:** Read-write

- **Kernel guarantee:** Every variable in BSS is **zero-initialized** (arithmetic types → 0; pointers → NULL) by the kernel before the program begins execution
- **Efficiency:** BSS does not store actual zero values in the binary file — it only records the size; actual zeroing happens at load time (reduces executable size)

Example. 💡 Example:

```
int global_uninitialized;    // BSS — automatically 0
static int s;               // BSS — automatically 0
int arr1000];              // BSS — all elements 0
```

- 3.5 Stack Memory
- **Mechanism:** Uses the **stack data structure** (LIFO — Last In, First Out)
- **Contents per function call (stack frame):**
 - Local variables
 - Function parameters (passed by value)
 - Return address (instruction to execute after function returns)
 - Saved register values
- **Growth direction:** Grows **downward** (from high to low addresses), in the direction **opposite to the heap**
- **Management:** Automatic — the compiler inserts push/pop instructions; no manual management needed
- **Scope:** Memory is valid only within the function's execution scope; automatically reclaimed on return

Example. 💡 Example:

```
void foo(int x) {
    int local = 5;    // Stored on stack frame of foo()
    // When foo() returns, this frame is popped — local & x no longer
    accessible
}
```

Stack Overflow occurs when the stack grows beyond its allocated limit (e.g., infinite recursion).

- 3.6 Heap Memory

- **Nature:** Dynamic — not fixed at compile time; grows and shrinks at **runtime**
- **Contents:** Memory explicitly allocated during program execution via malloc, calloc, realloc
- **Growth direction:** Grows **upward** (from low to high addresses), opposite to stack
- **Management: Manual** — programmer is responsible for allocating (malloc/calloc) and deallocating (free)
- **Also called:** Free pool of memory, dynamic memory
- **Access:** Only accessible via **pointers** — there is no named variable for heap memory, only pointer references

Distinction: "Heap memory" in memory layout $\neq$ "Heap data structure". They share a name but are entirely different concepts.

§ 4. Memory Allocation in C

- **4.1 Compile-Time (Static) Memory Allocation**

- Memory is reserved for variables and constants **during compilation**, before the program runs
- Size is **fixed** — cannot be increased or decreased at runtime
- Requires knowing exact memory requirements in advance
- Also referred to as **static memory allocation** or **stack memory allocation**

Key Features:

Feature	Detail
Allocation/Deallocation	Automatic — handled by the compiler
Data structure used	Stack

Variable lifetime	For the duration of the scope in which they are declared
Size flexibility	Fixed; cannot be changed at runtime
Reusability	Memory cannot be reused after deallocation
Execution speed	Faster than dynamic allocation (no runtime overhead)
Memory efficiency	Less efficient — may over-allocate or under-allocate
Memory allocation timing	Before runtime (at compile time)

Example. ❖ Example:

```
int arr100];    // 100 × 4 = 400 bytes allocated at compile time
char name50];  // 50 × 1 = 50 bytes allocated at compile time
```

-
- 4.2 Run-Time (Dynamic) Memory Allocation
 - Memory is allocated **during program execution**
 - Heap is used as the backing store — referred to as **dynamic memory allocation**
 - Pointers are the **only** mechanism to access heap-allocated memory
 - Four library functions manage heap allocations: malloc(), calloc(), realloc(), free()
 - Requires #include <stdlib.h>

Key Features:

Feature	Detail
Allocation	At runtime using malloc/calloc

Deallocation	Explicit using free() — memory is NOT automatically reclaimed
Size flexibility	Can grow or shrink using realloc()
Reusability	Memory can be reused after free()
Memory efficiency	High — allocate only what is needed
Memory wastage	None if managed correctly; leaks possible if free() is omitted

- 4.3 malloc()

Full name: Memory Allocation

Purpose: Dynamically allocate a single contiguous block of memory of specified byte size.

Syntax:

```
ptr = (cast-type*) malloc(byte-size);
```

Behavior:

- Allocates byte-size bytes of contiguous memory from the heap
- Returns a void* pointer to the first byte of the allocated block
- If allocation fails (insufficient memory), returns NULL
- Initializes memory with **garbage values** (contents of previously freed memory — not zeroed)

Example.  Example:

```
#include <stdlib.h>
```

```
int *ptr;
```

```
// Case 1: Allocate space for 1 integer (4 bytes on most systems)
```

```
ptr = (int*) malloc(1 * sizeof(int));
```

```
// ptr now holds address of 4 bytes of heap memory
```



```
// Case 2: Allocate space for 100 integers (400 bytes)
ptr = (int*) malloc(100 * sizeof(int));
// ptr holds address of first byte in a 400-byte block
// ptr0], ptr1], ..., ptr99] are all accessible

// Always check for NULL before use
if (ptr == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}
```

- 4.4 calloc()

Full name: Contiguous Allocation

Purpose: Dynamically allocate multiple blocks of memory, each of a specified size, with **zero-initialization**.

Syntax:

```
ptr = (cast-type*) calloc(n, element-size);
```

Parameters:

- n — number of elements to allocate
- element-size — size (in bytes) of each element

Behavior:

- Allocates $n \times \text{element-size}$ bytes of contiguous memory
- Returns void* to the first byte; returns NULL on failure
- **Initializes all bytes to 0** (unlike malloc which leaves garbage values)

Example. 💎 Example:

```
float *ptr;
```

```
// Allocate contiguous space for 25 floats, all initialized to 0.0
```

```
ptr = (float*) calloc(25, sizeof(float));

if (ptr == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

// ptr0] through ptr24] are all 0.0f
```

- 4.5 realloc()

Full name: Re-Allocation

Purpose: Dynamically resize previously allocated memory (from malloc or calloc).

Syntax:

```
ptr = realloc(ptr, newSize);
```

Behavior:

- Resizes the memory block pointed to by ptr to newSize bytes
- If expanding AND adjacent free space exists → expands in-place (**Case 1**)
- If expanding AND no adjacent free space → allocates a new block, copies existing data, frees old block, returns new pointer (**Case 2**)
- If shrinking → reduces block size, truncating excess (**Case 3**)
- Existing data is preserved; newly added bytes (if expanding) contain **garbage values**
- Returns NULL if reallocation fails (original pointer remains valid in this case — do not overwrite it with NULL return)

Example. 💡 Example:

```
int *ptr = (int*) malloc(5 * sizeof(int)); // 20 bytes
```

```
// Expand to hold 10 integers (40 bytes)
```

```
ptr = realloc(ptr, 10 * sizeof(int));
```

```
// Shrink back to 3 integers (12 bytes)
```

```
ptr = realloc(ptr, 3 * sizeof(int));
```

```
free(ptr); // Always free when done
```

- 4.6 free()

Purpose: Release dynamically allocated heap memory back to the free pool.

Syntax:

```
free(ptr);
```

Behavior:

- Marks the block pointed to by ptr as available for future allocations
- Does NOT zero the memory — the pointer still holds the old address (a **dangling pointer**)
- **Critical:** Memory allocated by malloc/calloc/realloc is NOT automatically freed; free() must be called explicitly
- Calling free() on a NULL pointer is safe (no-op); calling on an already-freed pointer is **undefined behavior**

Best practice after free:

```
free(ptr);
```

```
ptr = NULL; // Prevent dangling pointer dereference
```

Example. 💡 Example:

```
int *ptr = (int*) malloc(100 * sizeof(int));
```

```
// ... use ptr ...
```

```
free(ptr); // Return 400 bytes to heap
```

```
ptr = NULL; // Safety: prevent dangling pointer
```

- 4.7 malloc() vs calloc() Comparison

Attribute	malloc()	calloc()
Full name	Memory Allocation	Contiguous Allocation
Parameters	1 — total byte size	2 — count (n) + element size
Syntax	malloc(n * sizeof(type))	calloc(n, sizeof(type))
Initialization	Garbage values (uninitialized)	All bytes set to 0
Return type	void*	void*
Speed	Slightly faster (no zeroing step)	Slightly slower (must zero memory)
Use case	When you'll immediately overwrite the memory	When zero-initialization is needed (e.g., arrays, counters)
Failure return	NULL	NULL

- 4.8 Static vs Dynamic Allocation Comparison

Attribute	Compile-Time (Static)	Run-Time (Dynamic)
When allocated	During compilation	During execution
Memory region	Stack (and data segment for globals/statics)	Heap
Size	Fixed; cannot change	Variable; can be changed with realloc()

Reusability	Cannot reuse	Can reuse after free()
Library functions	Not required	malloc(), calloc(), realloc(), free()
Resize at runtime	Not possible	Possible via realloc()
Management	Automatic (compiler)	Manual (programmer)
Speed	Faster	Slower (runtime overhead)
Efficiency	Less efficient (potential wastage)	More efficient (allocate exactly what's needed)
Risk	Stack overflow	Memory leaks, dangling pointers

§ 5. Arrays — Array-Based List Implementation

• 5.1 Definition & Properties

Definition. *Definition: An array is a data structure that stores a fixed-size sequential collection of elements of the same type in contiguous memory locations.*

Properties:

Property	Detail
Homogeneous	All elements must be of the same data type
Contiguous	Elements occupy adjacent memory locations
0-indexed	First element at index 0, last at index n-1

Random access	Any element accessible in O(1) via index
Fixed size	Size declared at compile time (static array)
Access formula	Address of <code>arr[i]</code> = Base Address + $i \times \text{sizeof}(\text{type})$

Example. ♦ Example: Array `numbers[5]` with base address 1000, int (4 bytes):

- `numbers[0]` → address 1000
- `numbers[1]` → address 1004
- `numbers[2]` → address 1008
- `numbers[3]` → address 1012
- `numbers[4]` → address 1016

Visual: Array memory layout

- 5.2 Array Operations — Algorithms & Code

IsEmpty

Algorithm `IsEmpty(LIST)`:

```

IF CurrentSize == 0:
    RETURN "LIST is Empty"
ELSE:
    RETURN "LIST is not Empty"

```

IsFull

Algorithm `IsFull(LIST)`:

```

IF CurrentSize == MaxSize:
    RETURN "LIST is FULL"
ELSE:
    RETURN "LIST is not FULL"

```

Insert at End

Algorithm:

Algorithm InsertAtEnd(LIST, element):

```
IF IsFull(LIST):  
    RETURN "List is full. Can't Insert"  
position ← CurrentSize + 1  
LIST[position - 1] ← element  
CurrentSize ← CurrentSize + 1
```

C Implementation:

```
#include <stdio.h>  
  
#define MAX 5  
  
int main() {  
    int list[MAX];  
    int currentSize, element, i;  
  
    printf("Enter current size: "); scanf("%d", &currentSize);  
    printf("Enter %d elements:\n", currentSize);  
    for (i = 0; i < currentSize; i++) scanf("%d", &list[i]);  
  
    if (currentSize == MAX) { printf("List is full. Can't Insert\n"); return 0;  
}  
  
    printf("Enter element to insert: "); scanf("%d", &element);  
    list[currentSize] = element;    // Position = currentSize + 1, index =  
currentSize  
    currentSize++;  
  
    printf("List after insertion:\n");
```

```
    for (i = 0; i < currentSize; i++) printf("%d ", list[i]);  
    return 0;  
}
```

Delete from End

Algorithm:

Algorithm DeleteFromEnd(LIST):

IF IsEmpty(LIST):

 RETURN "List is Empty. Can't Delete"

position ← CurrentSize

// Element at LISTposition-1] is logically deleted

CurrentSize ← CurrentSize - 1

C Implementation:

```
if (currentSize == 0) { printf("List is Empty. Can't Delete\n"); return 0; }  
// No shifting required — just decrement size  
currentSize--;  
// Elements 0 to currentSize-1 are now the valid list
```

Insert at Front

Algorithm:

Algorithm InsertAtFront(LIST, element):

IF IsFull(LIST):

 RETURN "List is full. Can't Insert"

// Shift all elements one position forward (right)

FOR i ← CurrentSize-1 DOWNT0 0:

 LIST[i+1] ← LIST[i]

LIST[0] ← element

CurrentSize $\leftarrow$ CurrentSize + 1

C Implementation:

```
if (currentSize == MAX) { printf("List is full. Can't Insert\n"); return 0; }  
printf("Enter element to insert at front: "); scanf("%d", &element);
```

```
// Shift right: from last element backwards to index 0
```

```
for (i = currentSize - 1; i >= 0; i--)
```

```
    list[i + 1] = list[i];
```

```
list[0] = element;
```

```
currentSize++;
```

Time Complexity: $O(n)$ — all elements shifted

Delete from Front

Algorithm:

Algorithm DeleteFromFront(LIST):

IF IsEmpty(LIST):

 RETURN "List is Empty. Can't Delete"

// Shift all elements one position backward (left)

FOR i $\leftarrow$ 1 TO CurrentSize-1:

 LIST[i-1] $\leftarrow$ LIST[i]

CurrentSize $\leftarrow$ CurrentSize - 1

C Implementation:

```
for (i = 1; i < currentSize; i++)
```

```
    list[i - 1] = list[i];
```

```
currentSize--;
```

Insert at nth Position

Algorithm:

Algorithm InsertAtN(LIST, element, n):

IF IsFull(LIST): RETURN "List is full. Can't Insert"

IF IsEmpty(LIST) AND n == 1:

LIST[0] ← element; CurrentSize ← 1; RETURN

IF n > CurrentSize + 1: RETURN "nth Position Not available in List"

// Shift elements from n to CurrentSize one position right

FOR i ← CurrentSize-1 DOWNTON n-1:

LIST[i+1] ← LIST[i]

LIST[n-1] ← element

CurrentSize ← CurrentSize + 1

C Implementation:

// Shift elements at positions n to currentSize rightward

for (i = currentSize - 1; i >= position - 1; i--)

list[i + 1] = list[i];

list[position - 1] = element;

currentSize++;

Delete from nth Position

Algorithm:

Algorithm DeleteAtN(LIST, n):

IF IsEmpty(LIST): RETURN "List is Empty"

IF n > CurrentSize OR n < 1: RETURN "nth Position Not available"

IF n == CurrentSize:

CurrentSize ← CurrentSize - 1 // Last element deleted, no shifting

ELSE:

```

FOR i ← n TO CurrentSize-1:
    LISTi-1] ← LISTi]      // Shift left from n+1 onward
CurrentSize ← CurrentSize - 1

```

C Implementation:

```

if (position == currentSize) {
    currentSize--;
} else {
    for (i = position; i < currentSize; i++)
        listi - 1] = listi];
    currentSize--;
}

```

Search in List

Algorithm:

```

Algorithm Search(LIST, searchElement):
    IF IsEmpty(LIST): RETURN "List is Empty"
    FOR position ← 1 TO CurrentSize:
        IF LISTposition-1] == searchElement:
            RETURN "Search Element available in Position " + position
    RETURN "Search Element not available in the List"

```

C Implementation:

```

int found = 0;
for (position = 1; position <= currentSize; position++) {
    if (listposition - 1] == searchElement) {
        printf("Search Element available in Position %d\n", position);
        found = 1;
        break;
    }
}

```

```

    }
}
if (!found) printf("Search Element not available in the List\n");

```

Traverse/Print List

Algorithm:

Algorithm Traverse(LIST):

IF IsEmpty(LIST): RETURN "List is Empty"

FOR position $\leftarrow$ 1 TO CurrentSize:

PRINT position, LIST[position-1]

Time Complexities — Array Operations:

Operation	Best Case	Worst Case	Average Case
Access by index	O(1)	O(1)	O(1)
Insert at end	O(1)	O(1)	O(1)
Delete from end	O(1)	O(1)	O(1)
Insert at front	O(n)	O(n)	O(n)
Delete from front	O(n)	O(n)	O(n)
Insert at nth position	O(1) (at end)	O(n) (at front)	O(n)
Delete at nth position	O(1) (at end)	O(n) (at front)	O(n)
Search (linear)	O(1)	O(n)	$O(n/2) = \mathbf{O(n)}$

§ 6. Linked Lists

• 6.1 Definition & Motivation

Definition. *Definition: A linked list is a linear, sequential-access data structure in which elements (nodes) are stored at non-contiguous memory locations, with each node containing data and a pointer to the next node in the sequence.*

Why this matters. Motivation — Limitations of Arrays:

- **Fixed size** — size must be known at compile time; allocated memory is fixed regardless of actual usage
- **Expensive insertion/deletion** — inserting or deleting at an arbitrary position requires shifting $O(n)$ elements
- **Contiguous memory requirement** — large arrays may fail if sufficient contiguous memory is unavailable

• 6.2 Linked List vs Array

Attribute	Array	Linked List
Memory layout	Contiguous	Non-contiguous
Size	Fixed (static)	Dynamic (grows/shrinks at runtime)
Random access	$O(1)$ — direct index	Not supported — must traverse from head
Binary search	Possible ($O(\log n)$)	Not applicable without random access
Insertion/Deletion at arbitrary position	$O(n)$ — requires shifting	$O(1)$ — pointer update (once position found by $O(n)$ traversal)
Memory overhead	Low (data only)	Higher (data + pointer per node)

Cache performance	Better (spatial locality)	Poor (scattered in memory)
Traversal direction	Bidirectional (by index ± 1)	Unidirectional (SLL) or Bidirectional (DLL)

- 6.3 Node Representation in C

A node in a linked list is represented using a struct containing at minimum two parts:

// Singly Linked List node

```
struct Node {
    int data;           // Data field — stores the value
    struct Node* next;  // Pointer field — stores address of next node
};
```

// Doubly Linked List node

```
struct Node {
    int data;
    struct Node* prev;  // Pointer to previous node
    struct Node* next;  // Pointer to next node
};
```

- A linked list is identified by a pointer to its **head** node
 - If head == NULL, the list is **empty**
 - The **last node's next pointer** is NULL in SLL/DLL; points back to head in circular variants
-

- 6.4 Types of Linked Lists

Type	Pointer(s) per Node	Last Node Points To	Traversal
Singly Linked List (SLL)	1 (next)	NULL	Forward only
Doubly Linked List (DLL)	2 (prev, next)	NULL (next), NULL (prev of head)	Forward & Backward
Circular Singly Linked List (CSLL)	1 (next)	head	Forward (cyclic)
Circular Doubly Linked List (CDLL)	2 (prev, next)	head (next) and head->prev (prev)	Forward & Backward (cyclic)

Visual: Singly Linked List

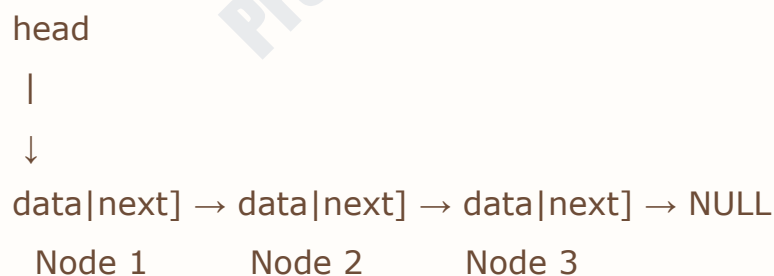
Visual: Doubly Linked List

Visual: Circular Singly Linked List

Visual: Circular Doubly Linked List

§ 7. Singly Linked List (SLL)

- 7.1 Structure & Pointer Model**



Definition. Node definition:

```

struct Node {
    int data;

```

```
    struct Node* next;
};
```

Dynamic node creation:

```
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (newNode == NULL) { printf("Memory allocation failed!\n"); exit(1);
}
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

- 7.2 Create & Display

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
};
```

```
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```



```
struct Node* insertAtEnd(struct Node* head, int value) {  
    struct Node* newNode = createNode(value);  
    if (head == NULL) return newNode;  
    struct Node* temp = head;  
    while (temp->next != NULL) temp = temp->next;  
    temp->next = newNode;  
    return head;  
}
```

```
void displayList(struct Node* head) {  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    struct Node* temp = head;  
    printf("Linked List: ");  
    while (temp != NULL) { printf("%d -> ", temp->data); temp =  
temp->next; }  
    printf("NULL\n");  
}
```

```
int main() {  
    struct Node* head = NULL;  
    int n, value;  
    printf("Enter number of nodes: "); scanf("%d", &n);  
    for (int i = 0; i < n; i++) {  
        printf("Enter data for node %d: ", i + 1); scanf("%d", &value);  
        head = insertAtEnd(head, value);  
    }  
    displayList(head);  
    return 0;  
}
```

- 7.3 Insert Operations

Insert at Beginning

Algorithm:

1. Create new node; set newNode->data = value
2. newNode->next = head (new node points to old head)
3. head = newNode (head now points to new node)

```
struct Node* insertAtBeginning(struct Node* head, int value) {  
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }  
    newNode->data = value;  
    newNode->next = head; // New node → old first node  
    head = newNode;      // Head updated to new node  
    return head;  
}
```

Example. ♦ Example: List: 10 → 20 → 30 → NULL, insert 5 at beginning:

- newNode->next = head (points to 10)
 - head = newNode (head now points to 5)
 - Result: 5 → 10 → 20 → 30 → NULL
-

Insert at End

```
struct Node* insertAtEnd(struct Node* head, int value) {  
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }  
    newNode->data = value;  
    newNode->next = NULL;  
    if (head == NULL) return newNode;    // Empty list — new node IS the  
list  
    struct Node* temp = head;  
    while (temp->next != NULL) temp = temp->next;    // Traverse to last  
node  
    temp->next = newNode;  
    return head;  
}
```

Time Complexity: $O(n)$ — must traverse to the last node

Insert at Any Position

```
struct Node* insertAtPosition(struct Node* head, int value, int position) {  
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (position == 1) {    // Insert at beginning  
        newNode->next = head;  
        return newNode;  
    }  
  
    struct Node* temp = head;
```

```

// Traverse to node at (position - 1)
for (int i = 1; i < position - 1 && temp != NULL; i++)
    temp = temp->next;

if (temp == NULL) { printf("Invalid Position!\n"); free(newNode);
return head; }

newNode->next = temp->next; // New node → node that was at
position
temp->next = newNode;      // Previous node → new node
return head;
}

```

Example. ♦ Example: Insert 15 at position 3 in 10 → 20 → 30 → NULL:

- Traverse to position 2 (node 20)
 - newNode->next = 20->next = node 30
 - 20->next = newNode
 - Result: 10 → 20 → 15 → 30 → NULL
-

- **7.4 Delete Operations**

Delete Front Node

```
struct Node* deleteFront(struct Node* head) {  
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return  
NULL; }  
    struct Node* temp = head;  
    printf("Deleted element: %d\n", temp->data);  
    head = head->next;    // Move head to second node  
    free(temp);          // Free old head node  
    return head;  
}
```

Delete Last Node

```
struct Node* deleteLast(struct Node* head) {  
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return  
NULL; }  
    if (head->next == NULL) {    // Only one node  
        printf("Deleted element: %d\n", head->data);  
        free(head); return NULL;  
    }  
    struct Node* temp = head;  
    struct Node* prev = NULL;  
    while (temp->next != NULL) { prev = temp; temp = temp->next; }  
    printf("Deleted element: %d\n", temp->data);  
    prev->next = NULL;    // Second-to-last node now points to NULL  
    free(temp);  
    return head;  
}
```

Delete at Any Position

```
struct Node* deleteAtPosition(struct Node* head, int position) {  
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return  
NULL; }  
    struct Node* temp = head;  
  
    if (position == 1) {          // Delete first node  
        printf("Deleted element: %d\n", temp->data);  
        head = temp->next;  
        free(temp); return head;  
    }  
  
    struct Node* prev = NULL;  
    for (int i = 1; i < position && temp != NULL; i++) { prev = temp;  
temp = temp->next; }  
  
    if (temp == NULL) { printf("Invalid position!\n"); return head; }  
  
    printf("Deleted element: %d\n", temp->data);  
    prev->next = temp->next;    // Bypass the node to delete  
    free(temp);  
    return head;  
}
```

Time Complexity – SLL Operations:

Operation	Time Complexity	Notes
Insert at beginning	O(1)	No traversal needed
Insert at end	O(n)	Must reach last node
Insert at position k	O(k)	Traverse k-1 nodes
Delete from beginning	O(1)	
Delete from end	O(n)	Must find second-to-last
Delete at position k	O(k)	
Search	O(n)	Linear scan
Traverse	O(n)	Visit all nodes

- 7.5 Search Operation

```
void search(struct Node* head, int key) {  
    struct Node* temp = head;  
    int position = 1, found = 0;  
    while (temp != NULL) {  
        if (temp->data == key) {  
            printf("Element %d found at position %d\n", key, position);  
            found = 1; break;  
        }  
        temp = temp->next;  
    }
```

```

        position++;
    }
    if (!found) printf("Element %d not found in the list.\n", key);
}

```

§ 8. Doubly Linked List (DLL)

● 8.1 Structure & Pointer Model

$$\begin{array}{ccccc}
 \text{NULL} & \leftarrow & \text{prev|data|next} & \Leftrightarrow & \text{prev|data|next} & \Rightarrow \\
 \text{head} & & & & & \text{tail} \\
 \text{prev|data|next} \rightarrow \text{NULL} & & & & &
 \end{array}$$

Each node maintains **two pointers**: prev (to predecessor) and next (to successor).

Advantages over SLL:

- Bidirectional traversal (forward and backward)
- Deletion of a node is **O(1)** if the node pointer is known (no need to find predecessor)

Disadvantage: Extra memory per node (prev pointer)

```

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

```

- 8.2 Create & Display (Forward/Backward)

// Insert at end for DLL

```
struct Node* insertAtEnd(struct Node* head, int value) {  
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }  
    newNode->data = value;  
    newNode->prev = NULL;  
    newNode->next = NULL;  
    if (head == NULL) return newNode;  
    struct Node* temp = head;  
    while (temp->next != NULL) temp = temp->next;  
    temp->next = newNode;  
    newNode->prev = temp; // KEY: set prev pointer of new node  
    return head;  
}
```

// Forward traversal

```
void displayForward(struct Node* head) {  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    struct Node* temp = head;  
    printf("Forward: ");  
    while (temp != NULL) { printf("%d <-> ", temp->data); temp =  
temp->next; }  
    printf("NULL\n");  
}
```

// Backward traversal

```
void displayBackward(struct Node* head) {  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    struct Node* temp = head;
```

```

while (temp->next != NULL) temp = temp->next; // Go to last node
printf("Backward: ");
while (temp != NULL) { printf("%d <-> ", temp->data); temp =
temp->prev; }
printf("NULL\n");
}

```

- 8.3 Insert Operations

Insert at Beginning (DLL)

```

struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL) head->prev = newNode; // Update old head's prev
    head = newNode;
    return head;
}

```

Insert at Any Position (DLL)

```

struct Node* insertAtPosition(struct Node* head, int value, int position) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) { printf("Memory allocation failed!\n"); exit(1); }
    newNode->data = value;

    if (position == 1) {
        newNode->prev = NULL;
        newNode->next = head;
    }
}

```

```

        if (head != NULL) head->prev = newNode;
        return newNode;
    }

    struct Node* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) temp =
temp->next;

    if (temp == NULL) { printf("Invalid Position!\n"); free(newNode);
return head; }

    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) temp->next->prev = newNode; // Update
next node's prev
    temp->next = newNode;
    return head;
}

```

Example. ♦ Example: Insert 15 at position 2 in 10 ⇌ 20 ⇌ 30:

- Traverse to node at position 1 (node 10)
 - newNode->next = 10->next = node 20
 - newNode->prev = 10
 - 20->prev = newNode
 - 10->next = newNode
 - Result: 10 ⇌ 15 ⇌ 20 ⇌ 30
-

- **8.4 Delete Operations**

- Delete Front Node (DLL)**

```
struct Node* deleteFront(struct Node* head) {  
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return  
NULL; }  
    struct Node* temp = head;  
    printf("Deleted element: %d\n", temp->data);  
    head = head->next;  
    if (head != NULL) head->prev = NULL; // New head's prev → NULL  
    free(temp);  
    return head;  
}
```

- Delete Last Node (DLL)**

```
struct Node* deleteLast(struct Node* head) {  
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return  
NULL; }  
    struct Node* temp = head;  
    if (temp->next == NULL) { printf("Deleted element: %d\n",  
temp->data); free(temp); return NULL; }  
    while (temp->next != NULL) temp = temp->next; // Reach last node  
    printf("Deleted element: %d\n", temp->data);  
    temp->prev->next = NULL; // Second-to-last's next → NULL  
    free(temp);  
    return head;  
}
```

Delete at Any Position (DLL)

```
struct Node* deleteAtPosition(struct Node* head, int position) {
    if (head == NULL) { printf("List is empty! Cannot delete.\n"); return
    NULL; }
    struct Node* temp = head;

    if (position == 1) {
        printf("Deleted element: %d\n", temp->data);
        head = temp->next;
        if (head != NULL) head->prev = NULL;
        free(temp); return head;
    }

    for (int i = 1; i < position && temp != NULL; i++) temp =
    temp->next;

    if (temp == NULL) { printf("Invalid position!\n"); return head; }

    printf("Deleted element: %d\n", temp->data);
    if (temp->next != NULL) temp->next->prev = temp->prev; // Next
    node's prev bypass
    if (temp->prev != NULL) temp->prev->next = temp->next; // Prev
    node's next bypass
    free(temp);
    return head;
}
```

Note. Key Insight: DLL deletion at a known node pointer is $O(1)$ because the prev pointer gives direct access to the predecessor — no need to re-traverse. SLL requires $O(n)$ traversal to find predecessor.

- 8.5 Search Operation

```
void search(struct Node* head, int key) {  
    struct Node* temp = head;  
    int position = 1, found = 0;  
    while (temp != NULL) {  
        if (temp->data == key) {  
            printf("Element %d found at position %d\n", key, position);  
            found = 1; break;  
        }  
        temp = temp->next;  
        position++;  
    }  
    if (!found) printf("Element %d not found in the list.\n", key);  
}
```

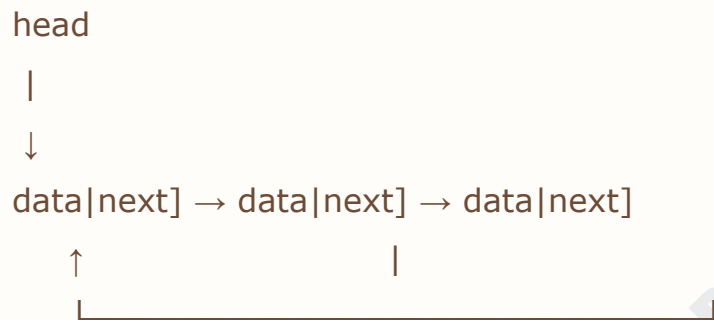
Prepared By Francis Neuben R

§ 9. Circular Singly Linked List (CSLL)

● 9.1 Structure & Invariant

Definition. *Definition: A circular singly linked list is a variant of SLL in which the last node's next pointer points back to the head node instead of NULL.*

Structural invariant: For any non-empty CSLL, `last_node->next == head`.



Visual: Circular Singly Linked List

Traversal: Uses a do-while loop (not while), because the stopping condition (`ptr == head`) is checked **after** the first visit, ensuring head is processed:

```
ptr = head;
do {
    // process ptr
    ptr = ptr->next;
} while (ptr != head);
```

Advantages:

- Efficient traversal — no NULL check needed; circular structure enables continuous cycling
- No NULL pointers in the list (reduces null-pointer dereference errors)
- Useful for repetitive/cyclic tasks (round-robin scheduling, token ring networks)

- Uniform traversal — can start from any node and reach every other node

Disadvantages:

- Complex implementation — must carefully maintain circular invariant during all modifications
- **Infinite loop risk** — a bug in traversal termination condition causes infinite looping
- Harder to debug — cycle-related bugs are difficult to detect
- Deletion complexity — requires updating both the predecessor's next and potentially head

Applications: Round-robin scheduling, multi-player games (turn rotation), circular queues, traffic light control, image/slideshow viewers, token passing in ring topology networks

- 9.2 Create & Display

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node { int data; struct node *next; };
```

```
struct node *head = NULL, *temp, *newnode;
```

```
void create() {
```

```
    int n, i;
```

```
    printf("Enter number of nodes: "); scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++) {
```

```
        newnode = (struct node*) malloc(sizeof(struct node));
```

```
        printf("Enter data for node %d: ", i); scanf("%d",
&newnode->data);
```

```
        if (head == NULL) {
```

```
            head = newnode; temp = newnode;
```

```
            newnode->next = head;           // Self-circular for first node
```

```
        } else {
```



```
        temp->next = newnode;
        newnode->next = head;           // Maintain circular link
        temp = newnode;
    }
}
}
```

```
void display() {
    struct node *ptr;
    if (head == NULL) { printf("List is empty\n"); return; }
    ptr = head;
    printf("Circular Linked List:\n");
    do {
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    } while (ptr != head);
    printf("(Back to Head)\n");
}
```

- **9.3 Insert Operations**

- Insert at Beginning (CSLL)**

- Algorithm:**

1. Create newNode
2. If list empty: newNode->next = newNode; head = newNode
3. Else:
 - a. Traverse to last node (temp->next != head)
 - b. newNode->next = head (new node → old head)
 - c. temp->next = newNode (last node → new node)
 - d. head = newNode (head updated)

```
void insert_beginning(int value) {
    struct node *newnode, *temp;
    newnode = (struct node*) malloc(sizeof(struct node));
    newnode->data = value;
    if (head == NULL) { head = newnode; newnode->next = head; }
    else {
        temp = head;
        while (temp->next != head) temp = temp->next; // Find last node
        newnode->next = head;
        temp->next = newnode; // Last node → new node (maintains
circle)
        head = newnode;
    }
}
```

Insert at End (CSLL)

```
void insert_end(int value) {
    struct node *newnode, *temp;
    newnode = (struct node*) malloc(sizeof(struct node));
    newnode->data = value;
    if (head == NULL) { head = newnode; newnode->next = head; }
    else {
        temp = head;
        while (temp->next != head) temp = temp->next; // Find last node
        temp->next = newnode;
        newnode->next = head; // New last node → head
    }
}
```

Insert at Any Position (CSLL)

```
void insert_position(int value, int pos) {
    struct node *newnode, *temp;
    int i;
    newnode = (struct node*) malloc(sizeof(struct node));
    newnode->data = value;
    if (pos == 1) { // Insert at beginning
        if (head == NULL) { head = newnode; newnode->next = head; }
        else {
            temp = head;
            while (temp->next != head) temp = temp->next;
            newnode->next = head; temp->next = newnode; head =
newnode;
        }
    } else {
        temp = head;
```

```

    for (i = 1; i < pos - 1; i++) temp = temp->next;
    newnode->next = temp->next;
    temp->next = newnode;
}
}

```

- 9.4 Delete Operations

Delete at Beginning (CSLL)

Three cases:

- Empty list → error
- Only one node → free; head = NULL
- Multiple nodes → find last, move head, update last→next = new head

```

void deleteBeginning() {
    struct Node *temp, *last;
    if (head == NULL) { printf("\nList is empty.\n"); return; }
    if (head->next == head) { // Only one node
        temp = head; head = NULL; free(temp);
        printf("\nOnly node deleted.\n"); return;
    }
    last = head;
    while (last->next != head) last = last->next; // Find last
    temp = head;
    head = head->next;
    last->next = head; // Last node's next → new head
    free(temp);
    printf("\nFirst node deleted.\n");
}

```

Delete at End (CSLL)

Three cases:

- Empty list → error
- Only one node → free; head = NULL
- Multiple nodes → traverse to second-to-last; set prev->next = head; free last

```
void deleteEnd() {
    struct Node *curr, *prev;
    if (head == NULL) { printf("\nList is empty.\n"); return; }
    if (head->next == head) { free(head); head = NULL; printf("\nOnly
node deleted.\n"); return; }
    prev = NULL; curr = head;
    while (curr->next != head) { prev = curr; curr = curr->next; }
    prev->next = head;           // Second-to-last → head (re-circles)
    free(curr);
    printf("\nLast node deleted.\n");
}
```

Delete at Any Position (CSLL)

```
void deleteAtPosition(int pos) {
    struct Node *temp, *prev, *last;
    int count = countNodes(), i;
    if (head == NULL) { printf("\nList is empty.\n"); return; }
    if (pos <= 0 || pos > count) { printf("\nInvalid position.\n"); return; }
    if (pos == 1) { deleteBeginning(); return; }
    // General case: traverse to position
    prev = head;
    for (i = 1; i < pos - 1; i++) prev = prev->next;
    temp = prev->next;
    prev->next = temp->next;
```

```
    free(temp);  
    printf("\nNode at position %d deleted.\n", pos);  
}
```

- 9.5 Search Operation

```
void search(int key) {  
    struct Node* temp;  
    int position = 1, found = 0;  
    if (head == NULL) { printf("\nList is empty.\n"); return; }  
    temp = head;  
    do {  
        if (temp->data == key) {  
            printf("\nElement %d found at position %d.\n", key, position);  
            found = 1; break;  
        }  
        temp = temp->next;  
        position++;  
    } while (temp != head);  
    if (!found) printf("\nElement %d not found.\n", key);  
}
```

§ 10. Circular Doubly Linked List (CDLL)

• 10.1 Structure & Invariant

Definition. *Definition: A circular doubly linked list combines the bidirectional linking of DLL with the circular structure of CSLL.*

Invariants:

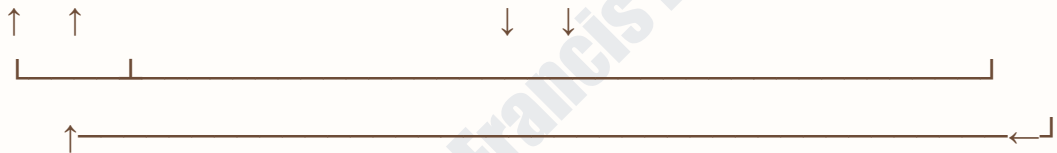
- `last_node->next == head`
- `head->prev == last_node`
- Every node has valid prev and next pointers (no NULLs)

head

|

↓

prev|data|next] ⇌ prev|data|next] ⇌ prev|data|next]



Key property: `head->prev` is a **direct O(1) reference to the last node** — no traversal needed.

Node structure:

```
struct Node {  
    int data;  
    struct Node* next;  
    struct Node* prev;  
};
```

• 10.2 Insert Operations

Insert at Beginning (CDLL)

```
void insertBeginning(int data) {
```

```
struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
if (!newNode) { printf("Memory allocation failed.\n"); return; }  
newNode->data = data;
```

```
if (head == NULL) {           // Empty list: self-circular  
    newNode->next = newNode;  
    newNode->prev = newNode;  
    head = newNode; return;  
}
```

```
    struct Node* last = head->prev;    // O(1) — direct access to last  
node  
    newNode->next = head;  
    newNode->prev = last;  
    head->prev = newNode;  
    last->next = newNode;  
    head = newNode;  
}
```


Insert at End (CDLL)

```
void insertEnd(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) { printf("Memory allocation failed.\n"); return; }
    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode; newNode->prev = newNode; head =
newNode; return;
    }

    struct Node* last = head->prev;    // Direct O(1) access to last
    newNode->next = head;
    newNode->prev = last;
    last->next = newNode;
    head->prev = newNode;
}
```

Insert at Any Position (CDLL)

```
void insertAtPosition(int data, int pos) {
    struct Node *newNode, *temp, *last;
    int count = countNodes(), i;
    newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) { printf("Memory allocation failed.\n"); return; }
    newNode->data = data;

    // Empty list
    if (head == NULL) {
        if (pos != 1) { printf("Invalid position.\n"); free(newNode); return;
}
}
```

```
        newNode->next = newNode; newNode->prev = newNode; head =  
newNode; return;
```

```
    }
```

```
        if (pos <= 0 || pos > count + 1) { printf("Invalid position.\n");  
free(newNode); return; }
```

```
// Insert at beginning
```

```
if (pos == 1) {
```

```
    last = head->prev;
```

```
    newNode->next = head; newNode->prev = last;
```

```
    last->next = newNode; head->prev = newNode;
```

```
    head = newNode; return;
```

```
}
```

```
// Insert at end
```

```
if (pos == count + 1) {
```

```
    last = head->prev;
```

```
    newNode->next = head; newNode->prev = last;
```

```
    last->next = newNode; head->prev = newNode; return;
```

```
}
```

```
// Insert at middle
```

```
temp = head;
```

```
for (i = 1; i < pos - 1; i++) temp = temp->next;
```

```
newNode->next = temp->next;
```

```
newNode->prev = temp;
```

```
temp->next->prev = newNode;
```

```
temp->next = newNode;
```

```
}
```

- 10.3 Delete Operations

Delete at Beginning (CDLL)

```
void deleteBeginning() {  
    struct Node *temp, *last;  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    if (head->next == head) { free(head); head = NULL; return; } //
```

One node

```
    temp = head;  
    last = head->prev;  
    head = head->next;  
    head->prev = last;  
    last->next = head;  
    free(temp);  
}
```

Delete at End (CDLL)

```
void deleteEnd() {  
    struct Node *last, *secondLast;  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    if (head->next == head) { free(head); head = NULL; return; }  
    last = head->prev;          // O(1): direct access via head->prev  
    secondLast = last->prev;  
    secondLast->next = head;  
    head->prev = secondLast;  
    free(last);  
}
```

Delete at Any Position (CDLL)

Six cases handled:

- Empty list → error
- Invalid position → error
- Only one node → free; head = NULL
- Delete at beginning → use deleteBeginning logic
- Delete at end → use deleteEnd logic
- Delete at middle → bypass node, update both prev and next of neighbors

```
void deleteAtPosition(int pos) {
```

```
    struct Node *temp, *last;
```

```
    int count = countNodes(), i;
```

```
    if (head == NULL) { printf("List is empty.\n"); return; }
```

```
    if (pos <= 0 || pos > count) { printf("Invalid position.\n"); return; }
```

```
    if (head->next == head) { free(head); head = NULL; return; } //
```

One node

```
    if (pos == 1) { // Delete beginning
```

```
        temp = head; last = head->prev;
```

```
        head = head->next; head->prev = last; last->next = head;
```

```
    free(temp); return;
```

```
    }
```

```
    if (pos == count) { // Delete end
```

```
        temp = head->prev; temp->prev->next = head; head->prev =  
temp->prev; free(temp); return;
```

```
    }
```

```
    // Delete middle
```

```
    temp = head;
```

```
    for (i = 1; i < pos; i++) temp = temp->next;
```

```
    temp->prev->next = temp->next;
```

```
    temp->next->prev = temp->prev;
```

```
    free(temp);  
}
```

- **10.4 Search Operation**

```
void search(int key) {  
    struct Node* temp;  
    int position = 1, found = 0;  
    if (head == NULL) { printf("List is empty.\n"); return; }  
    temp = head;  
    do {  
        if (temp->data == key) {  
            printf("Element %d found at position %d.\n", key, position);  
            found = 1; break;  
        }  
        temp = temp->next; position++;  
    } while (temp != head);  
    if (!found) printf("Element %d not found.\n", key);  
}
```

§ 11. Comparison Tables — All List Types

• Structure Comparison

Attribute	SLL	DLL	CSLL	CDLL
Pointers per node	1 (next)	2 (prev, next)	1 (next)	2 (prev, next)
Last node's next	NULL	NULL	head	head
Head's prev	N/A	NULL	N/A	last
Forward traversal				
Backward traversal				
NULL pointers in list	1 (last→next)	2 (head→prev, last→next)	0	0
O(1) access to last				(via head->prev)
Memory overhead	Low	Medium	Low	Medium
Implementation complexity	Low	Medium	Medium	High

• Operation Time Complexity Comparison

Operation	SLL	DLL	CSLL	CDLL
Insert at beginning	O(1)	O(1)	O(n)*	O(1)
Insert at end	O(n)	O(n)	O(n)	O(1)

Insert at position k	$O(k)$	$O(k)$	$O(k)$	$O(k)$
Delete at beginning	$O(1)$	$O(1)$	$O(n)^*$	$O(1)$
Delete at end	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete at known node	$O(n)$	$O(1)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Traverse	$O(n)$	$O(n)$	$O(n)$	$O(n)$

*CSLL insert/delete at beginning requires **$O(n)$** traversal to reach the last node to update its next pointer.

- When to Use Which

Use Case	Best Structure	Reason
Simple sequential storage	SLL	Minimal overhead
Need to traverse in both directions	DLL	$O(n)$ backward traversal
Round-robin scheduling, cyclic tasks	CSLL	Naturally models cycles
Need fast insert/delete at both ends + bidirectional traversal	CDLL	$O(1)$ at both ends via head->prev
Polynomial representation	SLL or DLL	Each node = one term
Undo/redo (browser history)	DLL	Forward + backward navigation

Music player playlist (repeat mode)

CSLL or
CDLL

Circular playback

§ 12. Applications & Polynomial Manipulation

● 12.1 Polynomial Representation

A **polynomial** is a mathematical expression of the form:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0 x^0$$

Example. ♦ Example: $P(x) = 4x^3 + 6x^2 + 7x + 9$

Each **term** has two attributes: coefficient and exponent.

Array Representation:

- Index represents exponent; value at index represents coefficient
- $P(x) = 4x^3 + 6x^2 + 7x + 9 \rightarrow 9, 7, 6, 4]$ (index 0 = x^0)
- **Problem:** Sparse polynomials waste enormous space. $P(x) = 16x^{21} - 3x^5 + 2x + 6$ requires an array of size 22 but uses only 4 slots.

Implement ation	Good For	Bad For
Array	Dense polynomials (most exponents used)	Sparse polynomials (few non-zero terms)
Linked List	Sparse polynomials	Backward traversal (SLL)

Linked List Node Structure:

```
struct polynomial {  
    int coefficient;  
    int exponent;  
    struct polynomial *next;  
};
```

// Alternative naming

```
struct polynode {
```



```

    int coef;
    int exp;
    struct polynode *next;
};
typedef struct polynode *polyptr;

```

Example. ♦ Example: $p_1(x) = 23x^9 + 18x^7 + 41x^6 + 163x^4 + 3$ stored as linked list:

23|9|→] → 18|7|→] → 41|6|→] → 163|4|→] → 3|0|→] → NULL

• 12.2 Polynomial Addition using Linked List

Algorithm (result stored in p3):

Compare current terms of p1 and p2. Three cases per comparison:

Case	Condition	Action
Case 1	$p1 \rightarrow \text{exp} > p2 \rightarrow \text{exp}$	Copy p1's node to p3; advance p1
Case 2	$p1 \rightarrow \text{exp} < p2 \rightarrow \text{exp}$	Copy p2's node to p3; advance p2
Case 3	$p1 \rightarrow \text{exp} == p2 \rightarrow \text{exp}$	Create new node with same exponent and summed coefficients; advance both

Continue until both lists are exhausted. Append remaining nodes of the non-exhausted list.

Example. ♦ Example:

p1: $4x^3 + 3x^2 + 2x$

p2: $2x^3 + 5x + 8$

Step 1: $4x^3$ vs $2x^3 \rightarrow$ exponents equal $\rightarrow (4+2)x^3 = 6x^3 \rightarrow p3: 6x^3]$

Step 2: $3x^2$ vs $5x^1 \rightarrow 2 > 1 \rightarrow$ copy $3x^2 \rightarrow p3: 6x^3 \rightarrow 3x^2]$

Step 3: $2x^1$ vs $5x^1 \rightarrow \text{equal} \rightarrow (2+5)x = 7x \rightarrow p3: 6x^3 \rightarrow 3x^2 \rightarrow 7x]$

Step 4: p1 exhausted; copy remaining p2 term 8 $\rightarrow p3: 6x^3 \rightarrow 3x^2 \rightarrow 7x \rightarrow 8]$

Result: $p3(x) = 6x^3 + 3x^2 + 7x + 8$

- 12.3 Polynomial Multiplication

Algorithm:

For each term t_i of p1 and each term t_j of p2:

- Create a new term with:
 - Coefficient = $t_i.\text{coef} \times t_j.\text{coef}$
 - Exponent = $t_i.\text{exp} + t_j.\text{exp}$
- Add this term to the result polynomial (combining like terms — same exponent $\rightarrow$ add coefficients)

Example.  Example:

p1: $2x^2 + 3x$

p2: $x + 4$

$$2x^2 \times x = 2x^3$$

$$2x^2 \times 4 = 8x^2$$

$$3x \times x = 3x^2$$

$$3x \times 4 = 12x$$

Result: $2x^3 + (8+3)x^2 + 12x = 2x^3 + 11x^2 + 12x$

Other Polynomial Operations:

- **Insertion:** Add a new term in sorted (descending exponent) order
- **Deletion:** Remove a term with a given exponent
- **Traversal:** Visit each term exactly once (forward only for SLL)
- **Merge sorted lists:** Standard merge operation based on exponent ordering

- **Differentiation:** For each term ax^n , produce $(a \times n)x^{n-1}$; constant term vanishes
- **Integration:** For each term ax^n , produce $a/(n+1)x^{n+1} + C$

Advantages of Linked List for Polynomials:

- No need to pre-allocate array size
- Sparse polynomials stored efficiently — only non-zero terms consume memory
- Easy insertion/deletion of terms

Disadvantages of Linked List for Polynomials:

- Cannot traverse backward (SLL)
- Cannot jump to the beginning from the end

§ Appendix: Quick Reference — Complete Complexity Summary

Data Structure	Insert Begin	Insert End	Insert at k	Delete Begin	Delete End	Search
Array	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$
SLL	$O(1)$	$O(n)$	$O(k)$	$O(1)$	$O(n)$	$O(n)$
DLL	$O(1)$	$O(n)$	$O(k)$	$O(1)$	$O(n)$	$O(n)$
CSLL	$O(n)$	$O(n)$	$O(k)$	$O(n)$	$O(n)$	$O(n)$
CDLL	$O(1)$	$O(1)$	$O(k)$	$O(1)$	$O(1)$	$O(n)$

§ Appendix: Common Pitfalls & Exam Tips

- **malloc returns void*** — always cast: `(int*) malloc(n * sizeof(int))`
- **Always check malloc/calloc return for NULL** before dereferencing
- **free() does not zero memory** — dangling pointer risk; set to NULL after freeing
- **CSLL traversal uses do-while**, not while — otherwise head node is skipped when stop condition is `ptr != head`
- **CDLL: head->prev always gives O(1) access to last node** — exploit this in insert/delete at end

- **Difference between BSS and Initialized Data Segment:** Both store globals/statics; BSS = uninitialized (auto-zeroed by kernel), Data = explicitly initialized
- **realloc on failure:** Returns NULL but original pointer remains valid — never do `ptr = realloc(ptr, size)` directly; use a temp: `temp = realloc(ptr, size); if(temp) ptr = temp;`
- **Stack grows downward, Heap grows upward** — they grow toward each other; collision = stack overflow or heap exhaustion
- **ADT vs Data Structure:** ADT = interface specification (what); Data Structure = implementation (how)
- **Array-based list insert/delete is $O(n)$** due to shifting, even though array access is **$O(1)$**

Notes are complete for Unit I (Abstract Data Types & Memory Allocation) and Unit II (Linked Lists). All content sourced from CS1006 Data Structures course material, RV University.

Prepared By Francis Mathew H